Integração com IIoT

Professor: Roberto

Aula 06

Cronograma Teórico

01 Introdução ao MQTT

02 Desafios

01

Introdução ao MQTT

MQTT

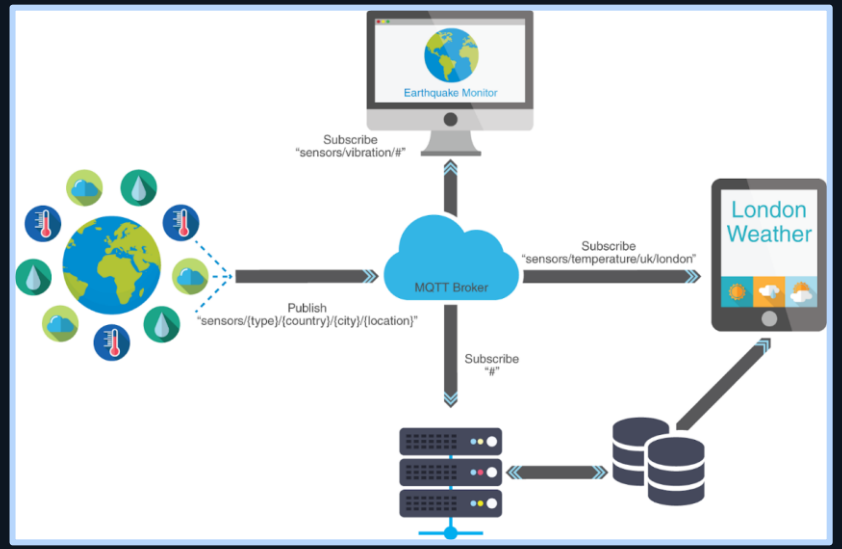
MQTT (Message Queuing Telemetry Transport) é um protocolo leve de comunicação projetado para dispositivos IoT, usado para enviar e receber mensagens entre dispositivos e servidores.

Características:

- Baseado no modelo publish/subscribe.
- Ideal para redes com largura de banda limitada ou dispositivos com recursos limitados.

Aplicações:

- Automação residencial.
- Monitoramento remoto.
- Sistemas industriais.



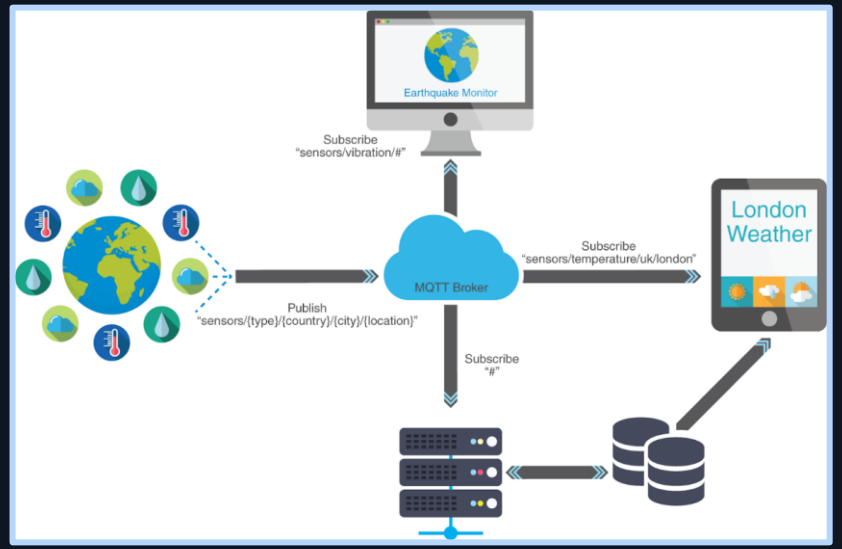
MQTT

Componentes Principais:

- **Broker:** Servidor central que gerencia as mensagens entre clientes.
- **Publisher:** Dispositivo que publica mensagens em um tópico.
- **Subscriber:** Dispositivo que recebe mensagens de um tópico.

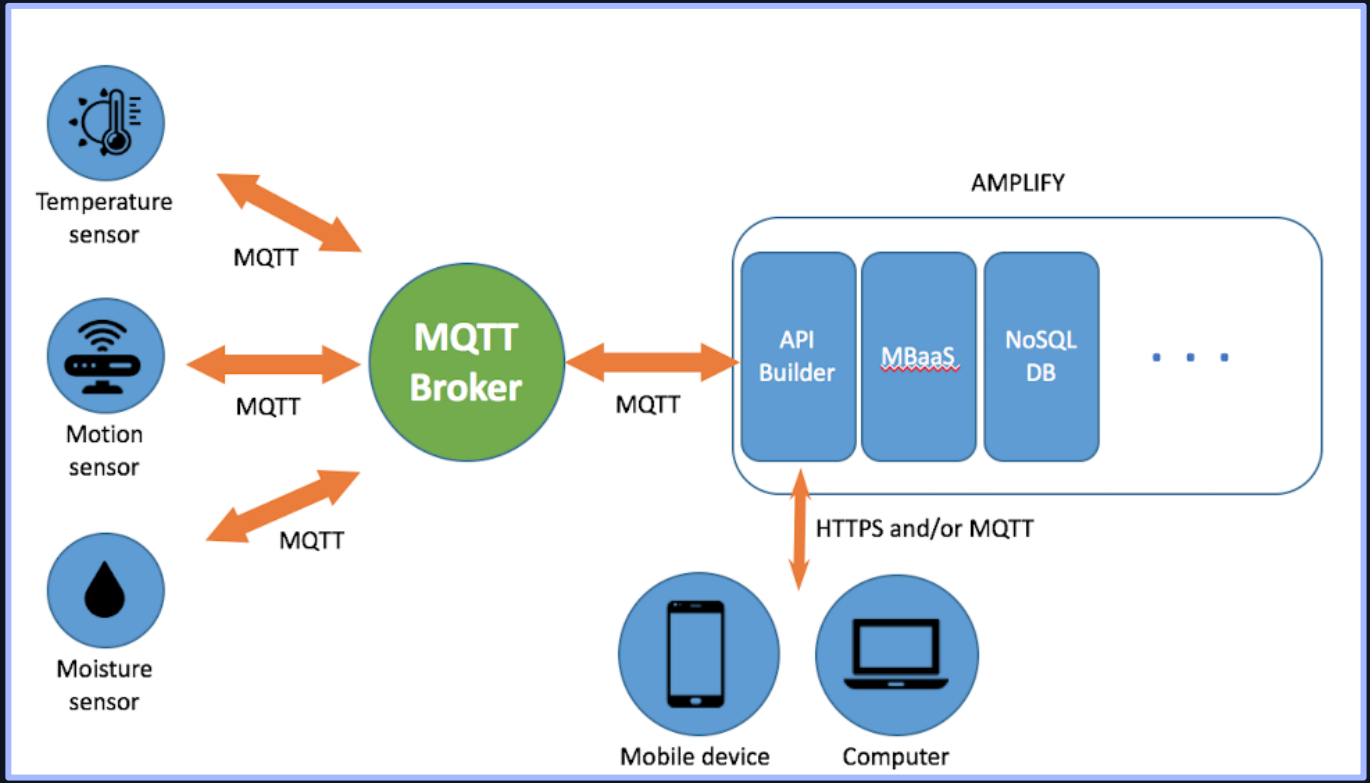
Fluxo:

- Publisher envia mensagens para o broker.
- Broker distribui as mensagens para os subscribers inscritos no tópico.



Fluxo de envio de dados via MQTT

ESP8266



MQTT

Tópicos são caminhos hierárquicos usados para organizar e direcionar mensagens no MQTT.

Exemplo de Tópicos:

- casa/luz/sala
- sensor/temperatura/cozinha

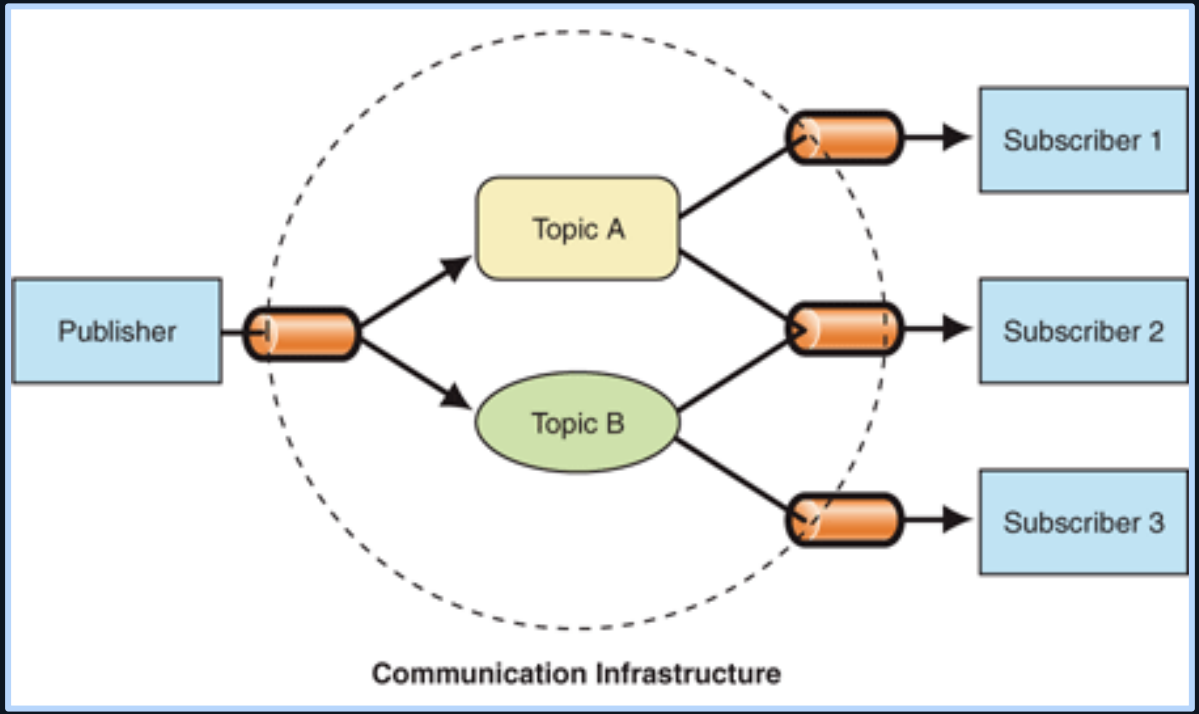
Hierarquia:
Use / para separar os níveis.

Exemplo:

- casa → nível superior.
- luz/sala → nível detalhado.

Fluxo de envio de dados via MQTT

ESP8266



MQTT - QoS

QoS define o nível de garantia para a entrega de mensagens no MQTT.

Níveis:

QoS 0 (At most once):

- Entrega sem garantias.
- Ideal para dados não críticos.

QoS 1 (At least once):

- Mensagem entregue pelo menos uma vez.
- Pode haver duplicatas.

QoS 2 (Exactly once):

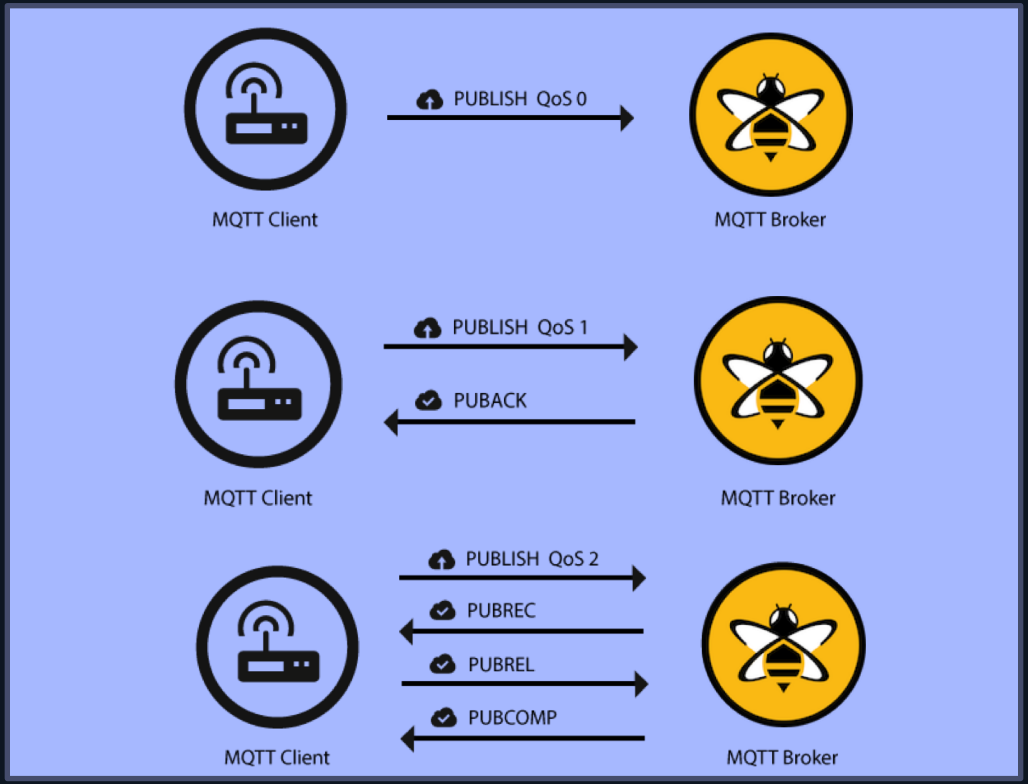
- Garantia de entrega única.
- Maior overhead.

Aplicações:

- Use QoS 0 para sensores.
- Use QoS 2 para dados críticos.

QoS - MQTT

ESP8266

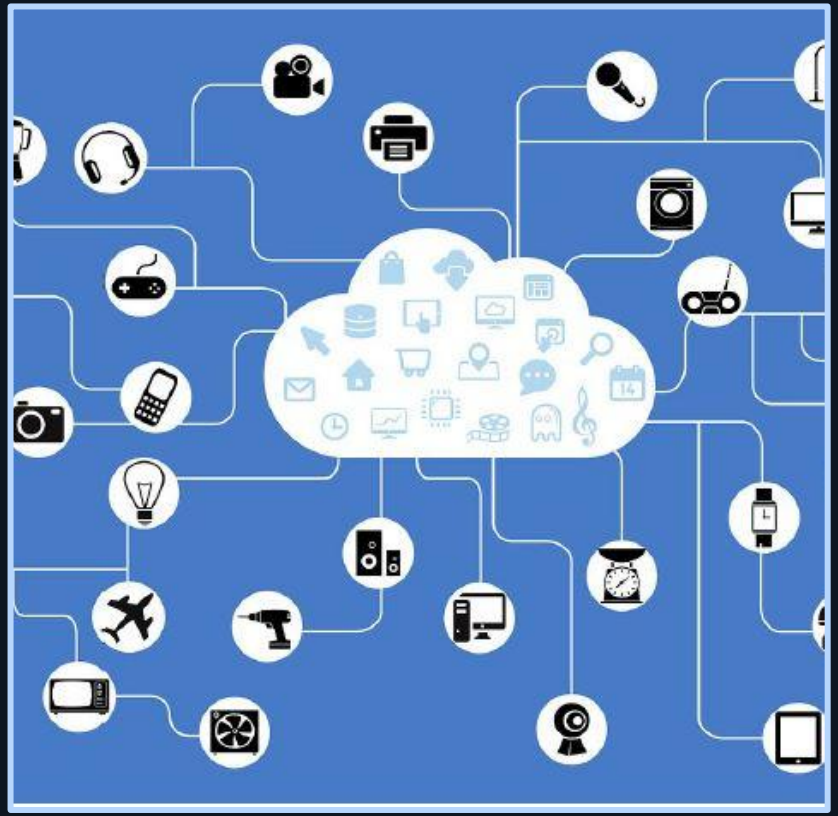


Plataforma Profissionais

Plataformas de MQTT oferecem serviços avançados para gerenciar a comunicação de dispositivos IoT, incluindo brokers, segurança e análise de dados.

Por que usar?

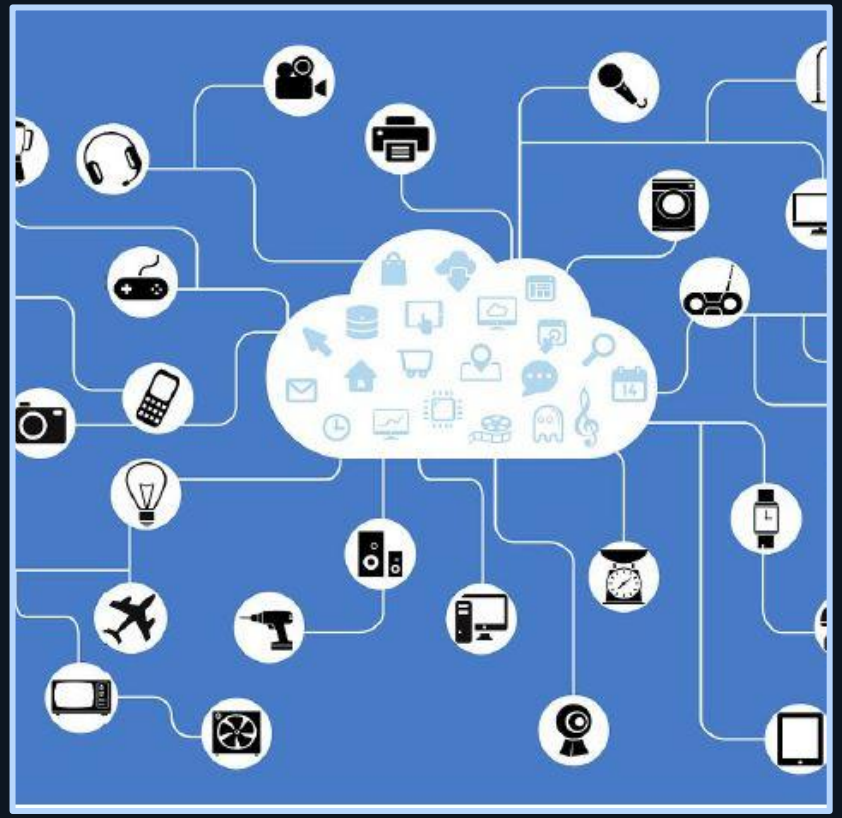
- Escalabilidade para milhares de dispositivos.
- Recursos de segurança integrados.
- Ferramentas para monitoramento e análise em tempo real.

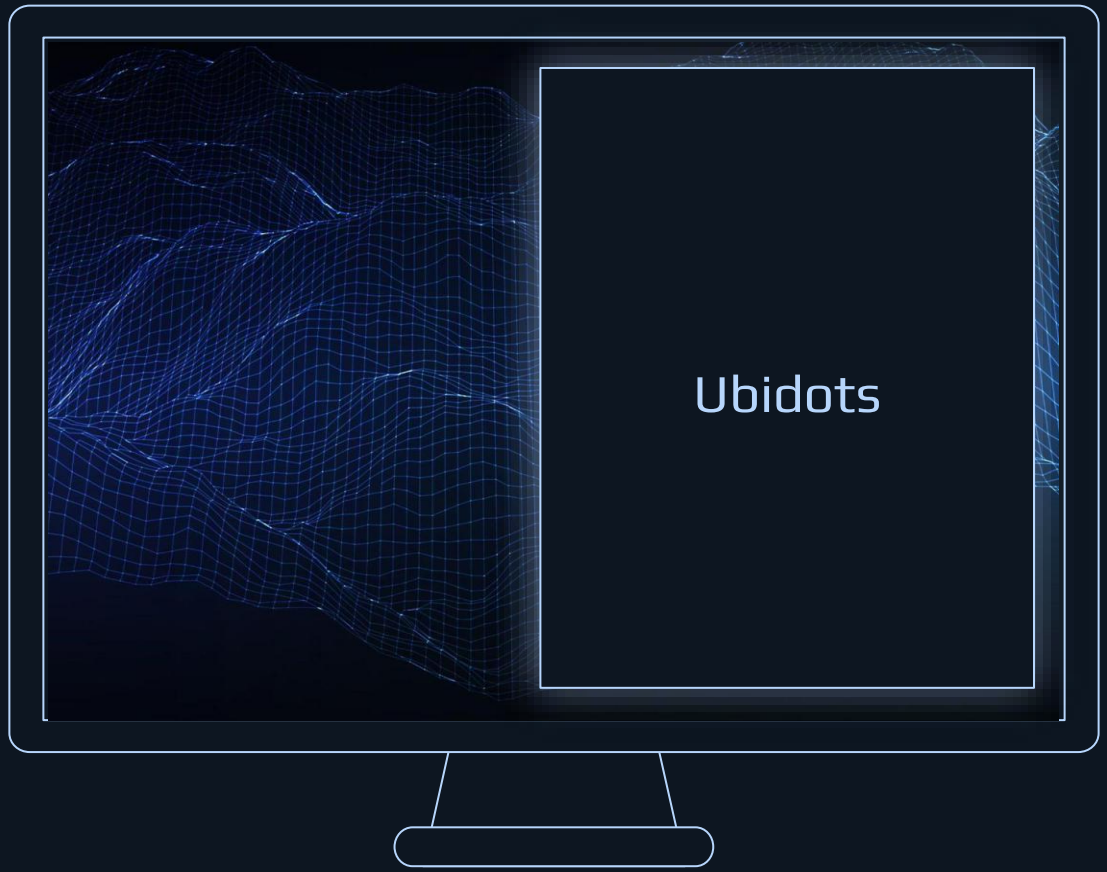


Plataforma Profissionais

Exemplos de plataformas:

- AWS IoT Core;
- HiveMQ;
- Adafruit IO;
- ThingsBoard;
- Google Cloud IoT Core;
- Azure IoT Hub;
- WMQX;
- Ubidots;
- Losant;
- Kaa IoT;
- Watson IoT Plataform;
- Particle;
- ThingSpeak
- TagoIO




Ubidots

Ubidots

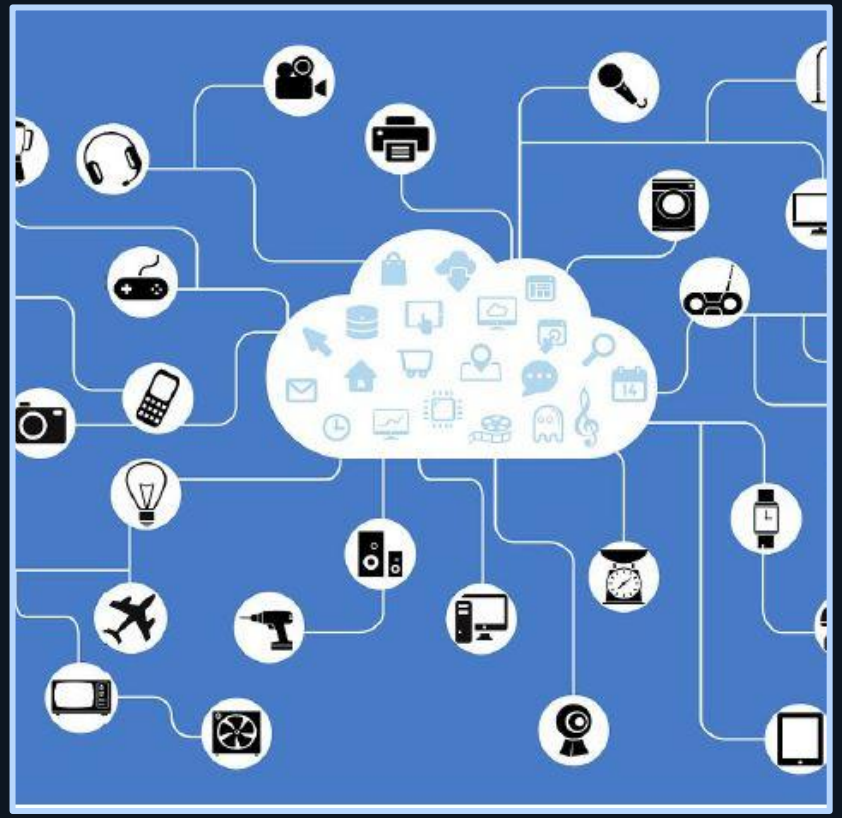
O Ubidots é uma plataforma de IoT que permite coletar, visualizar e gerenciar dados em tempo real de dispositivos conectados.

Características Principais:

- Suporte a MQTT, HTTP e Webhooks.
- Criação de dashboards interativos.
- Ideal para projetos de monitoramento e controle remoto.

Aplicações:

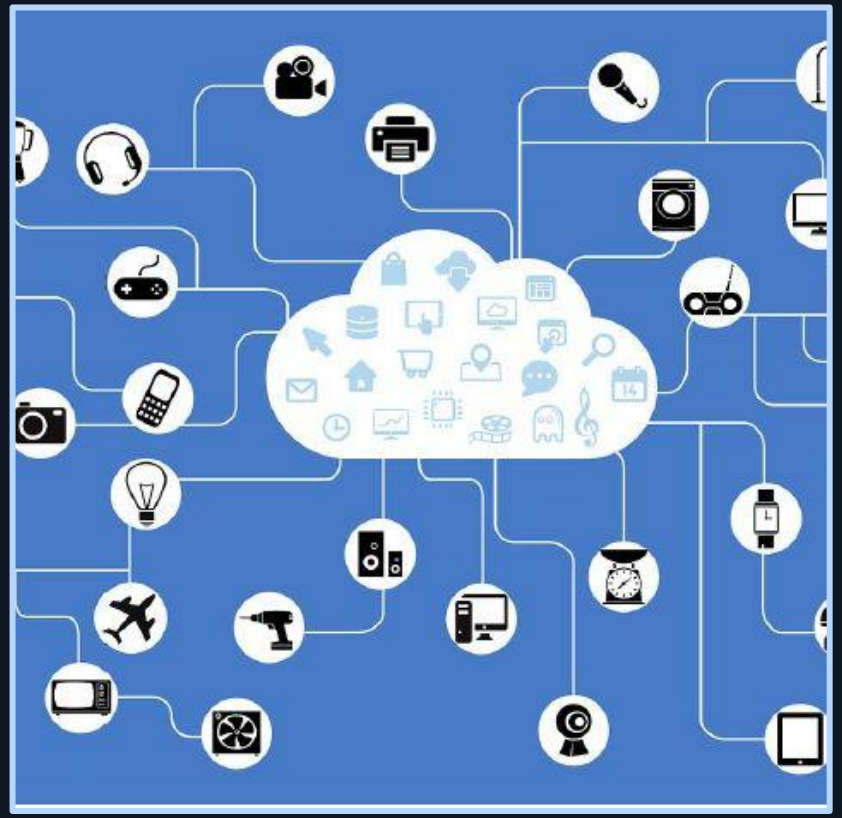
- Monitoramento ambiental.
- Controle de dispositivos em automação residencial e industrial.
- Projetos acadêmicos de IoT.



Ubidots

Porque usar o Ubidots:

- Fácil Integração: Compatível com ESP8266, ESP32, Arduino, Raspberry Pi, entre outros.
- Dashboards Interativos: Visualize dados com gráficos, medidores e indicadores.
- Plano Gratuito: Perfeito para testes e pequenos projetos.
- Segurança: Criptografia para comunicações seguras.

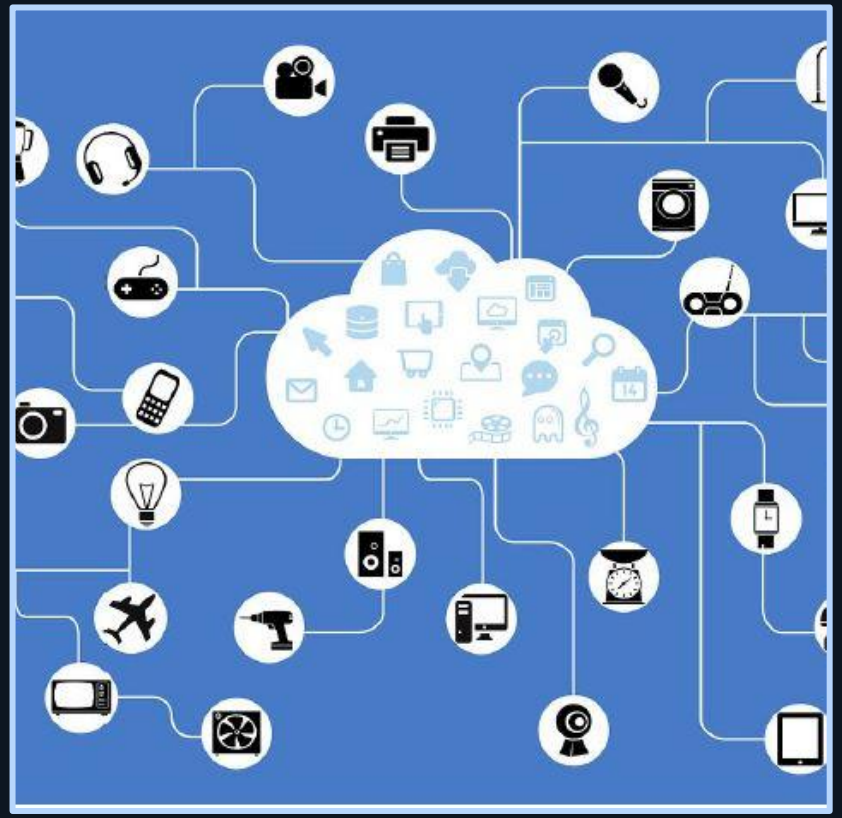


Ubidots

Criando uma Conta no Ubidots

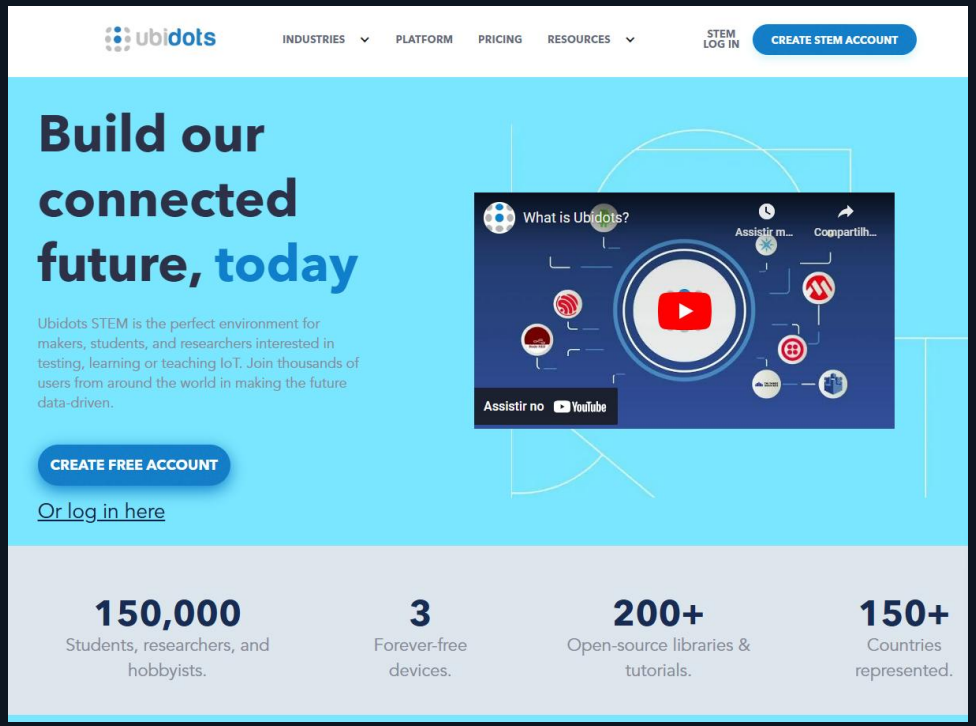
Passos Simples:

- Acesse https://ubidots.com/stem
- Crie uma conta gratuita.
- Faça login no painel principal.
- Não precisamos do plano profissional, o gratuito irá atender a nossa demanda.



Tela Ubidots

ESP8266

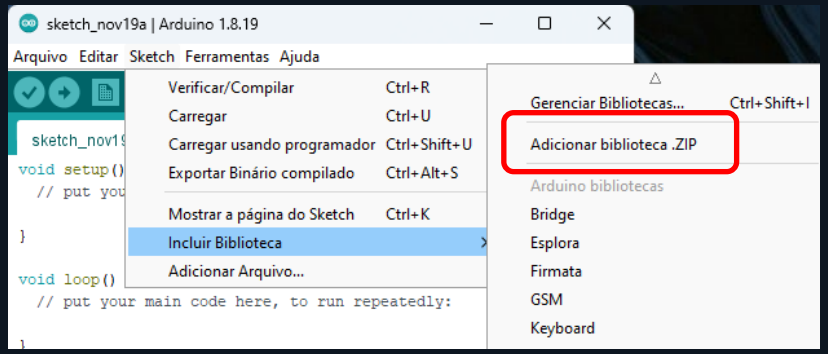


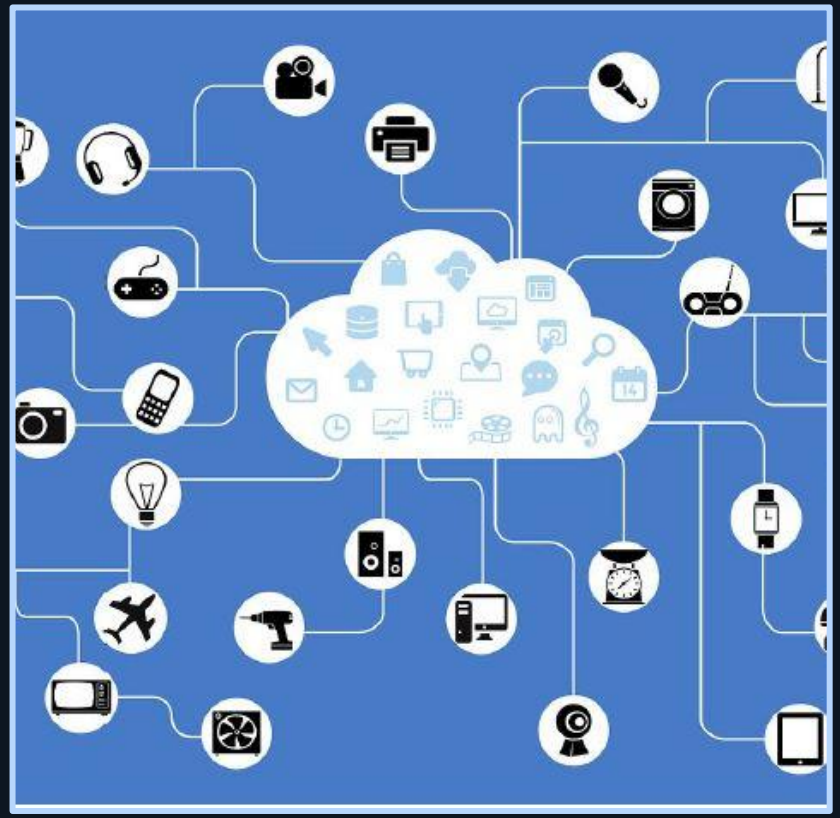
Ubidots

Depois da sua conta criada é hora de instalar a biblioteca para conexão com o Ubidots.

Faça o download do arquivo:
https://github.com/ubidots/ubidots-mqtt-esp.git

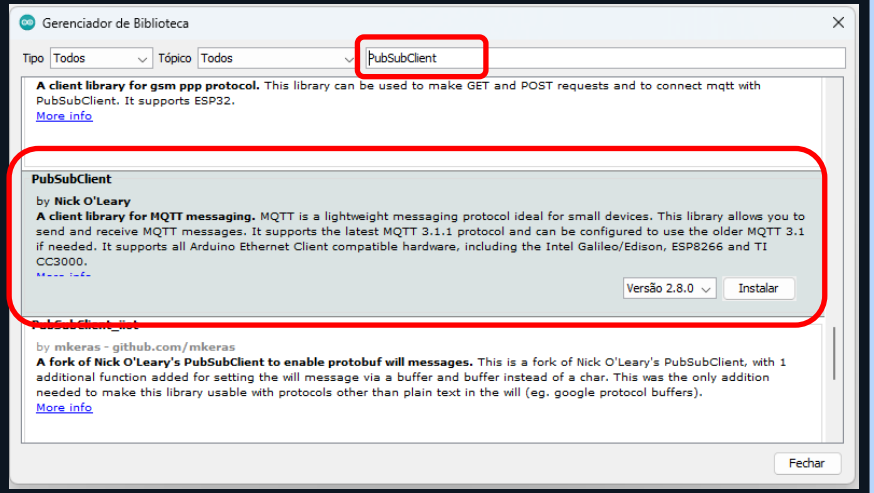
Vamos instalar ao arquivo:

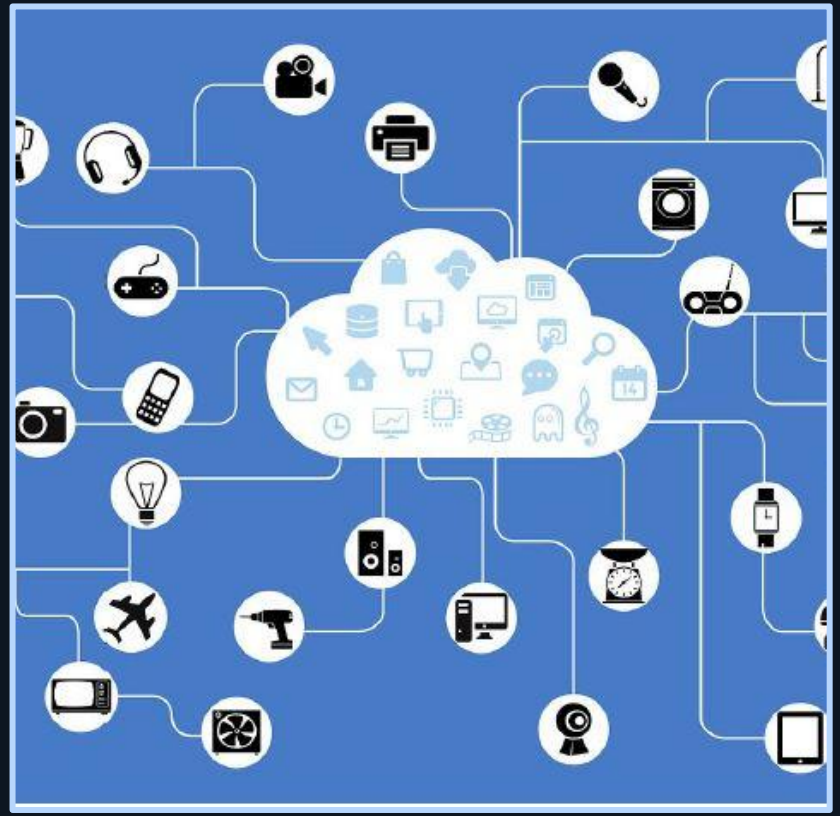




Ubidots

Além da biblioteca do ubidots, precisamos da PubSubClient



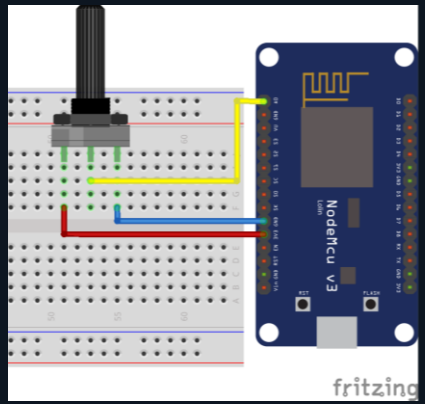


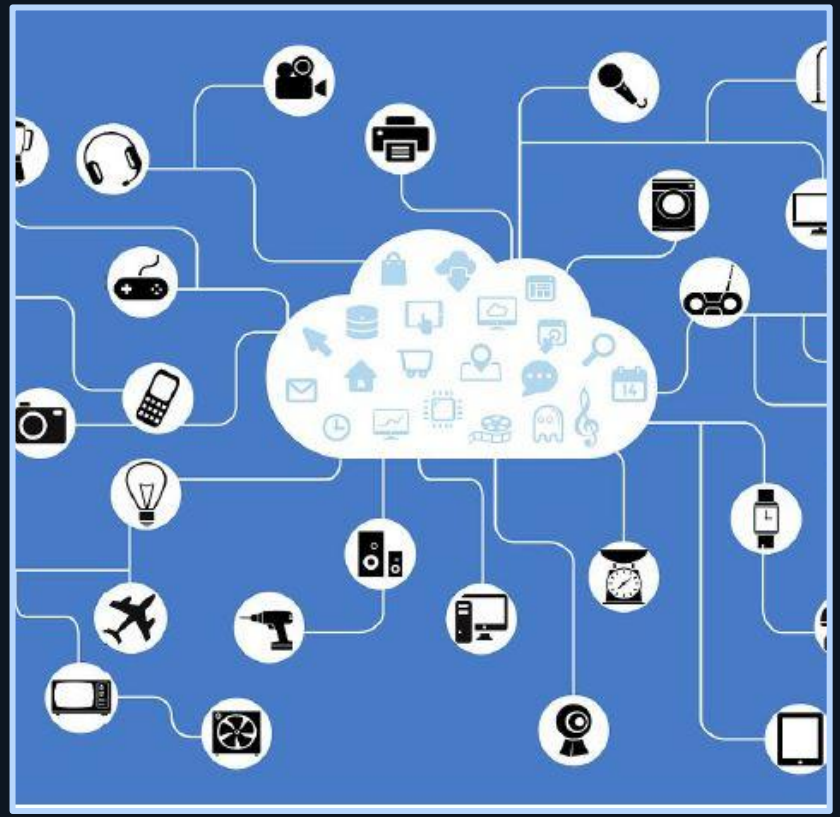
Ubidots

Vamos abrir o exemplo do código em:

Arquivo → Exemplos → Ubidots MQTT for ESP8266 → publish

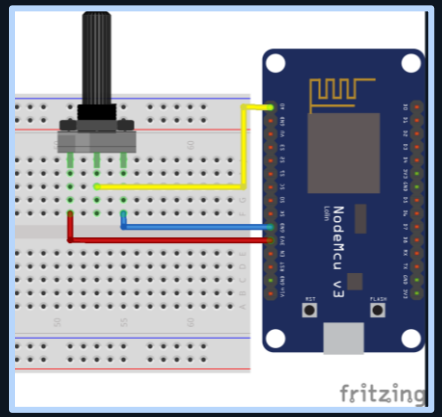
Vamos criar o primeiro programa: Vamos mandar um sinal analógico para o Ubidots através de um potenciômetro.





Ubidots

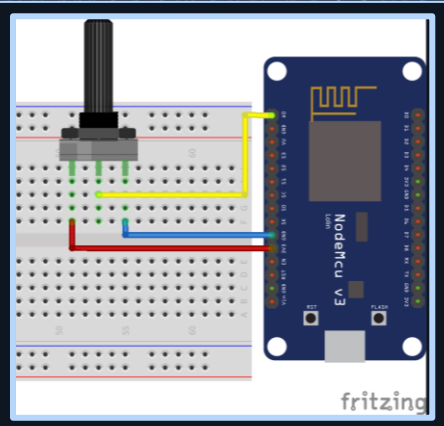
```
#include "UbidotsESPMQTT.h"

#define TOKEN “.......”
#define WIFINAME “.....”
#define WIFIPASS “.........”

Ubidots client(TOKEN);

void callback(char* topic, byte* payload, unsigned int length) {
  Serial.print("Message arrived [");
  Serial.print(topic);
  Serial.print("] ");
  for (int i = 0; i < length; i++) {
    Serial.print((char)payload[i]);
  }
  Serial.println();
}
```



A função callback em um código MQTT é usada para processar mensagens recebidas em tópicos aos quais o cliente MQTT está inscrito. Ela é essencial para tratar os dados que chegam ao dispositivo e definir ações baseadas nas mensagens.

Ubidots

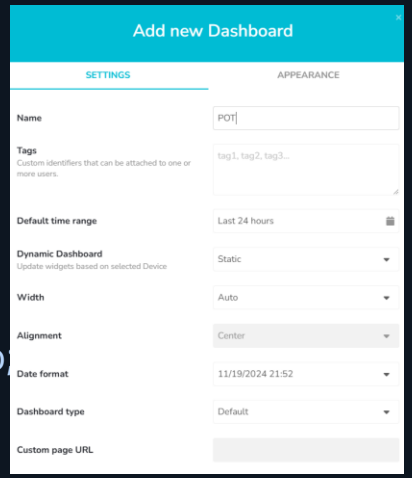
```
void setup() {
  Serial.begin(115200);
  client.setDebug(true);
  client.wifiConnection(WIFINAME, WIFIPASS);
  client.begin(callback);
}

void loop() {
  if (!client.connected()) {
    client.reconnect();
  }

  float pot = analogRead(A0);
  client.add("temperatura", pot);
  client.ubidotsPublish("potenciometro");
  client.loop();
}
```

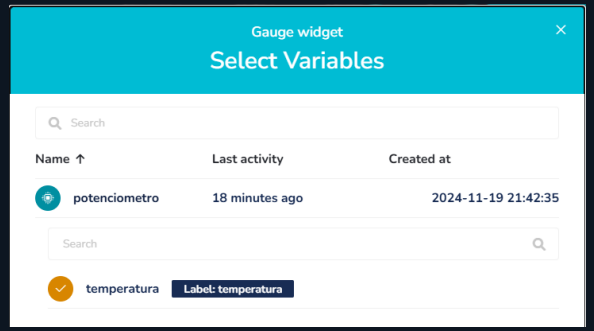


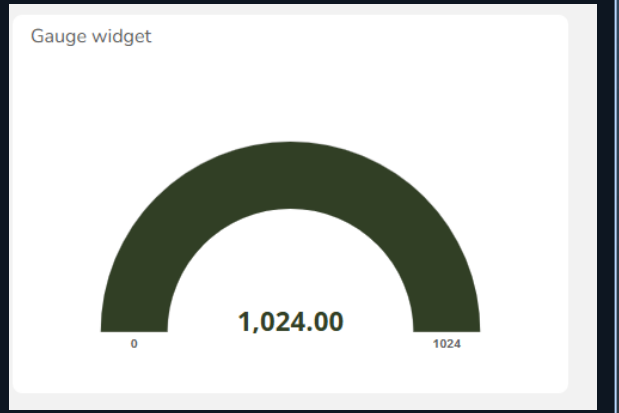
client.add("Topico","valor");
Client.ubidotsPublish("Dispositivo");
client.loop() → Mantém a conexão ativa.

Ubidots

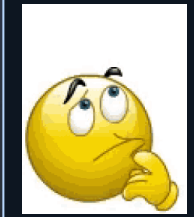
Criar dashboard:
Add New Dashboard;
Adicionar Widget;
Selecionar Gauge;
Adicionar variável;
Selecionar Potenciômetro;
Selecionar temperatura;







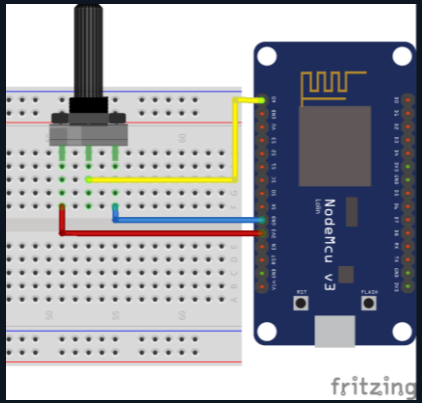
Qual o problema que vocês podem notar nesse código?

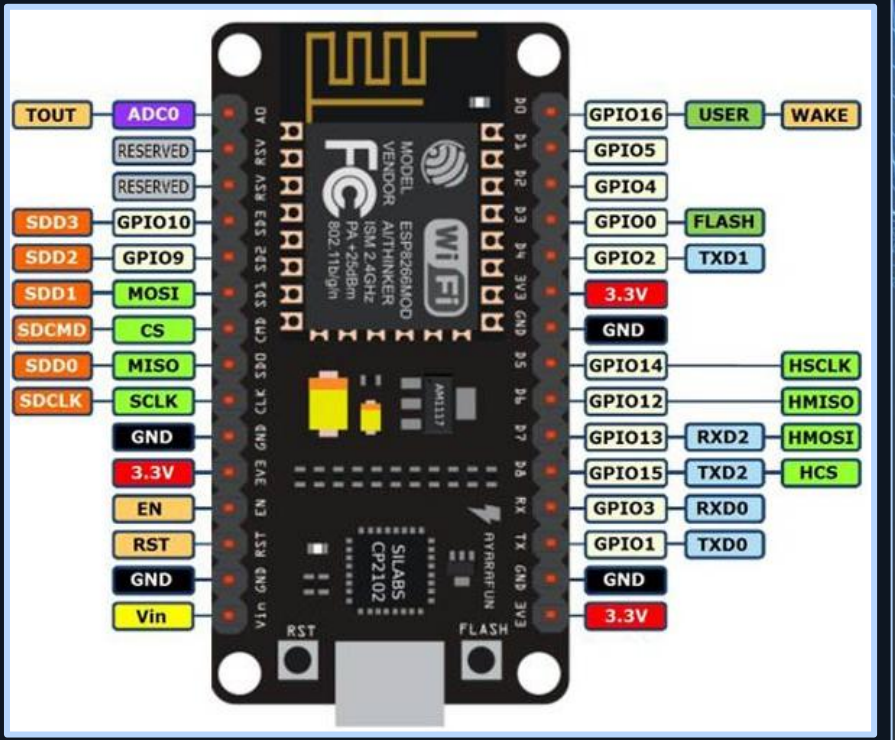


Ubidots

A taxa de conexão do ubidots é menor que a taxa de atualização do nosso micro controlador.

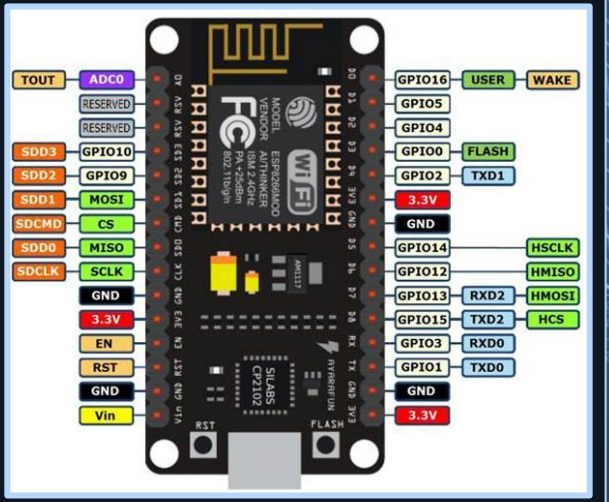
Faça um programa que o potenciômetro seja lido de 5 em 5 segundos.

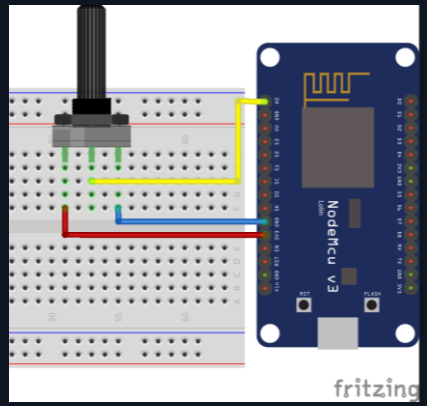




Ubidots

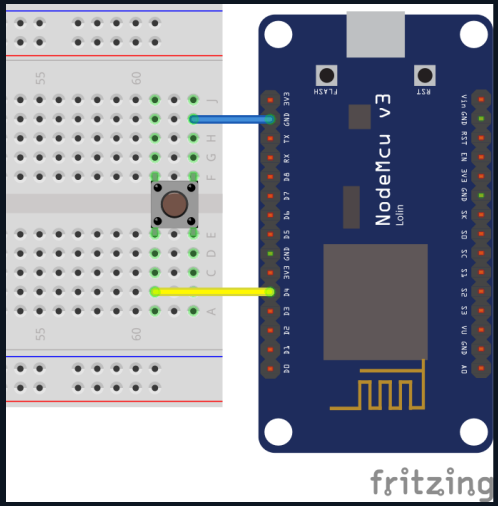
```
void loop() {
  if (!client.connected()) {
    client.reconnect();
  }

  if ((millis() - tempoatualizacao)>5000){
    float pot = analogRead(A0);
    client.add("temperatura", pot);
    client.ubidotsPublish("potenciometro");
    tempoatualizacao = millis();
  }

  client.loop();
}
```

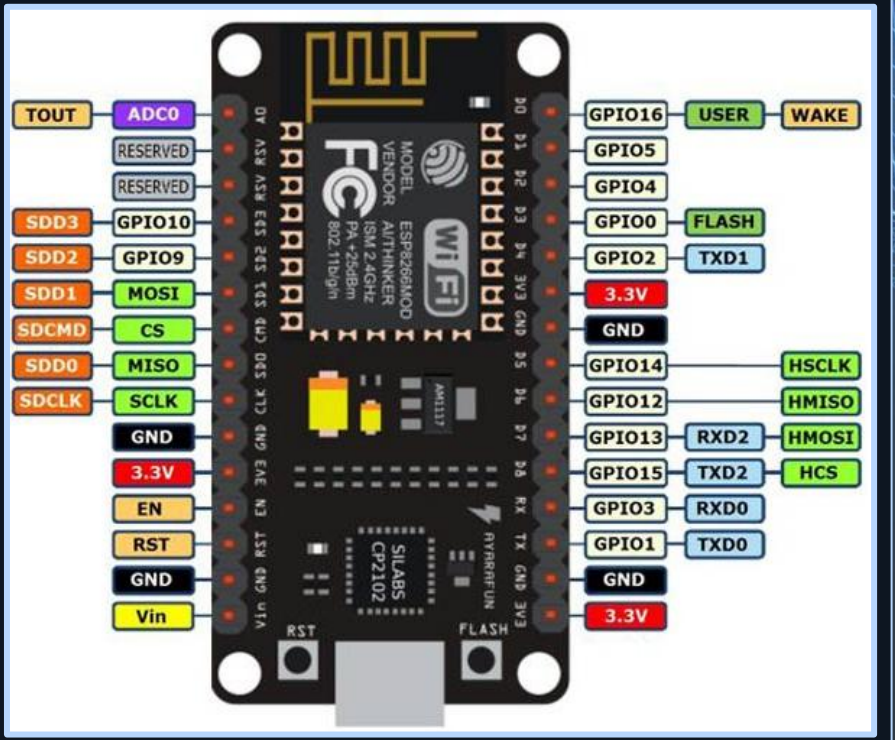




Ubidots

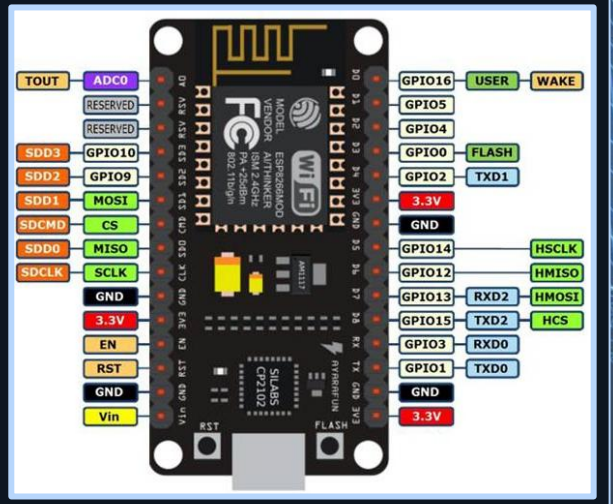
Faça um programa que a hora que pressionar um botão ligue um LED virtual, pressionou novamente o LED desliga.

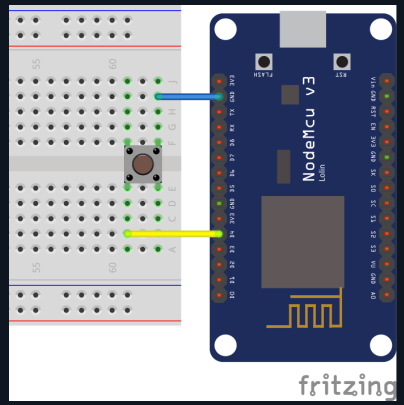


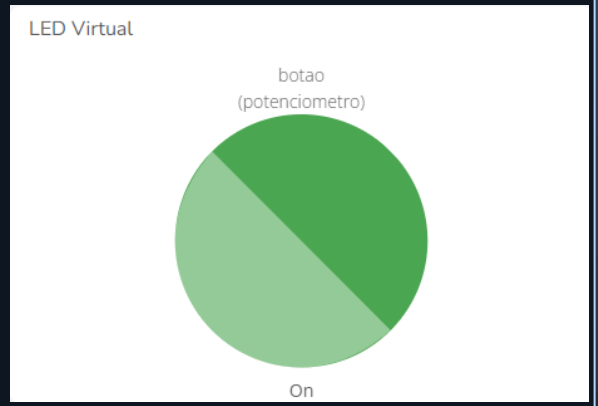


Ubidots

```
unsigned long debounce;
bool estado;

void loop() {
  if (!client.connected()) {
    client.reconnect();
  }

  if (digitalRead(2)==0){
    if ((millis() - debounce)>100){
    estado = !estado;
    client.add("botao", estado);
    client.ubidotsPublish("potenciometro");
  }
    debounce = millis();
  }
  client.loop();
}
```

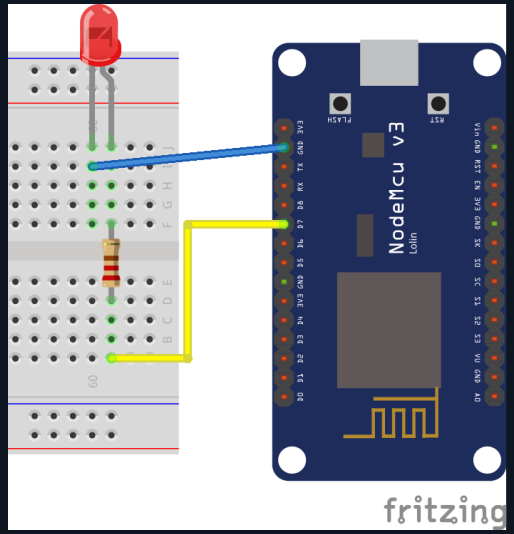


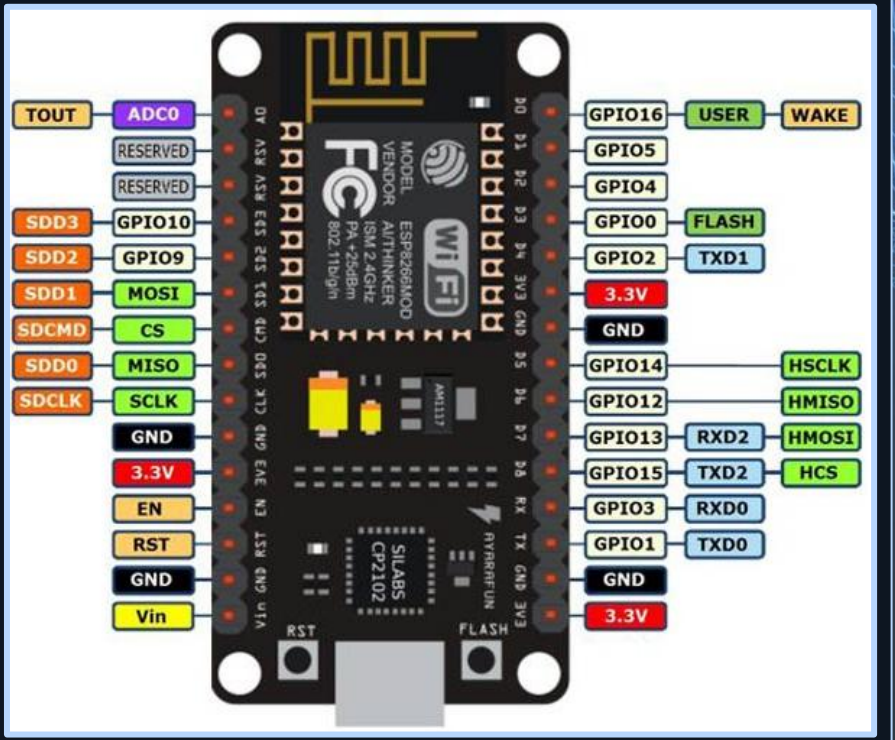




Ubidots

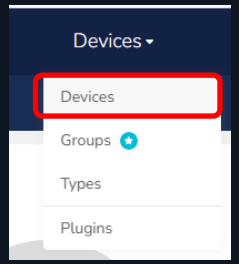
Faça um código onde um botão virtual ligue e desligue um LED físico

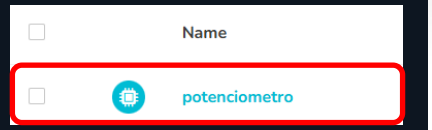


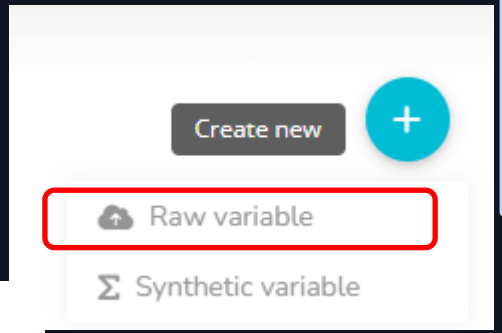


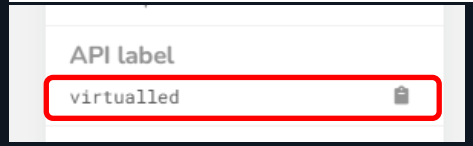
Ubidots

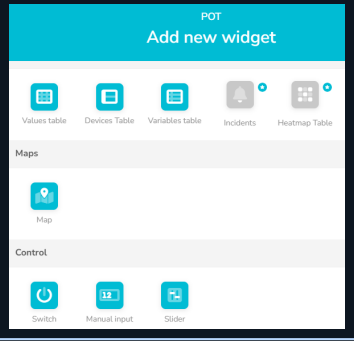
Antes de configurar o ESP8266 precisamos criar um tópico dentro do Ubidots para podermos receber o status do LED.

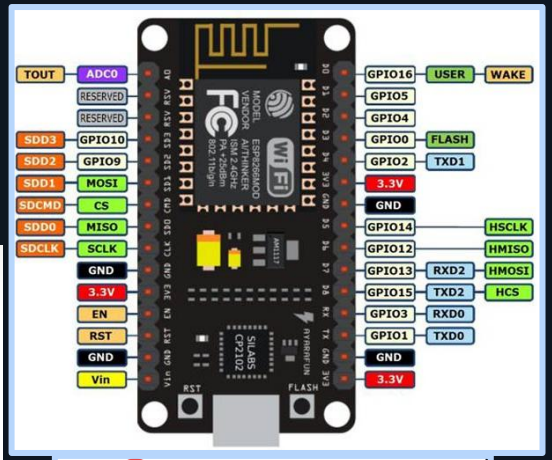


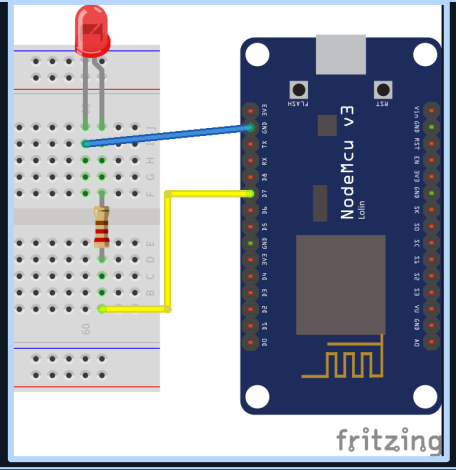






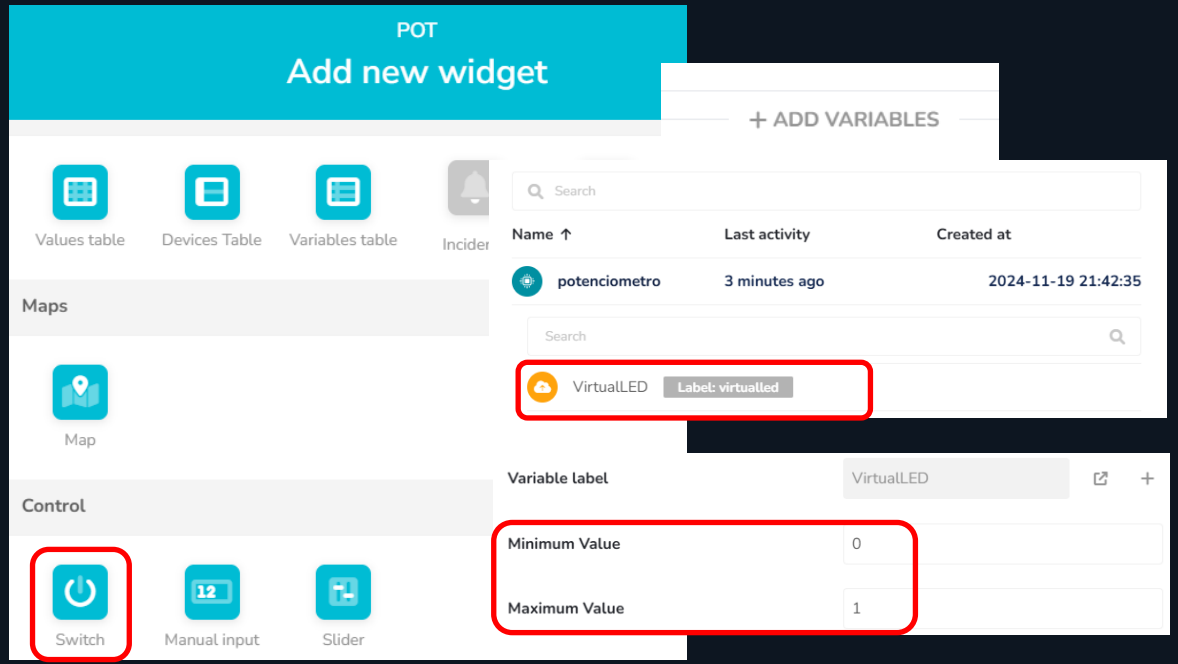


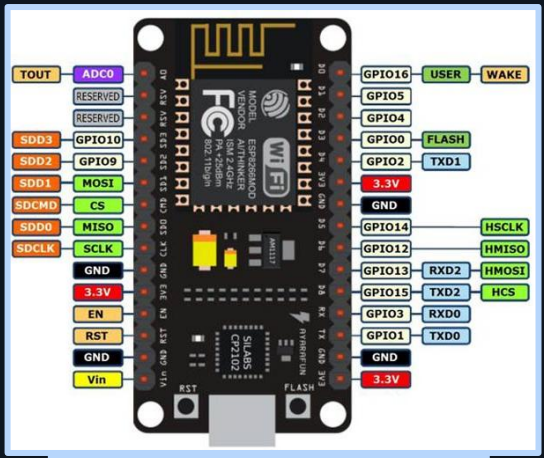


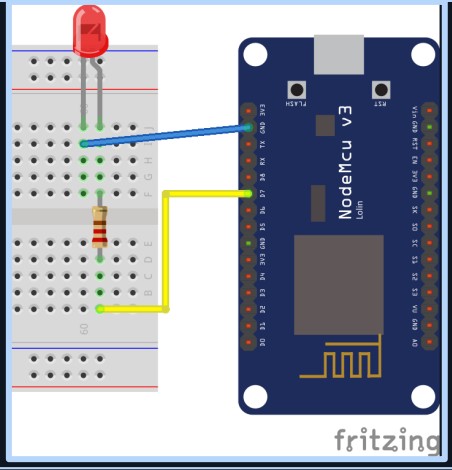


Ubidots

Agora vamos criar um botão no Dashboard,

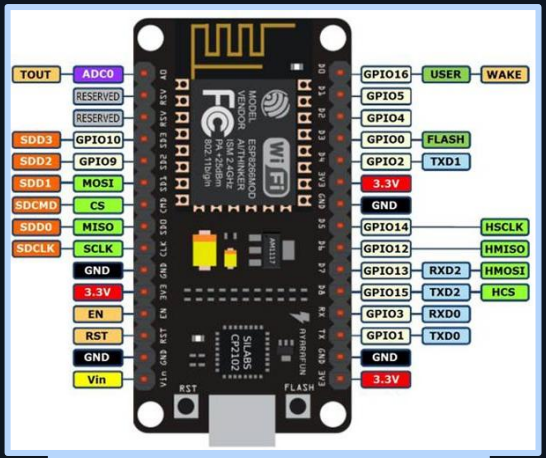


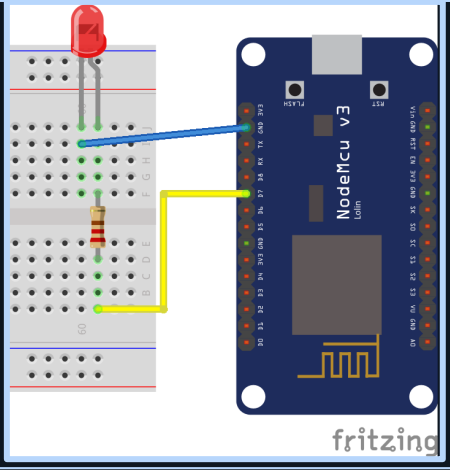




Ubidots

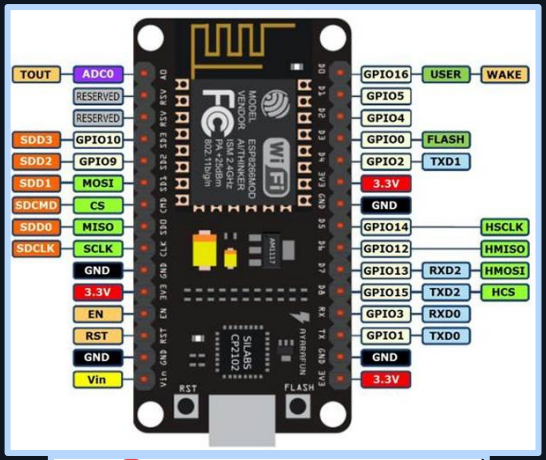
```
#define TOKEN “TOKEN"
#define WIFINAME “WIFI NAME"
#define WIFIPASS “SENHA WIFI"

#include "UbidotsESPMQTT.h"

Ubidots client(TOKEN);

const int ledPin = 13;
void callback(char* topic, byte* payload, unsigned int length) {
  Serial.print("Mensagem Recebida [");
  Serial.print(topic);
  Serial.print("] ");
  for (int i = 0; i < length; i++) {
    Serial.print((char)payload[i]);
  }
```

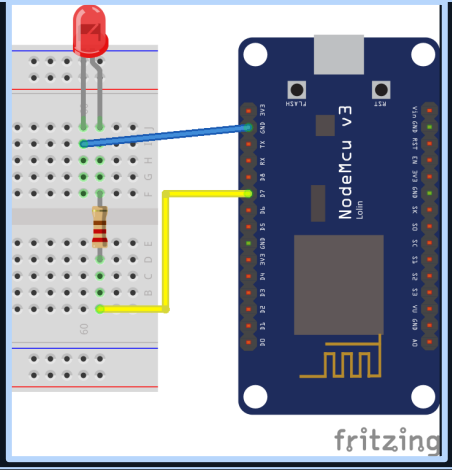




Ubidots

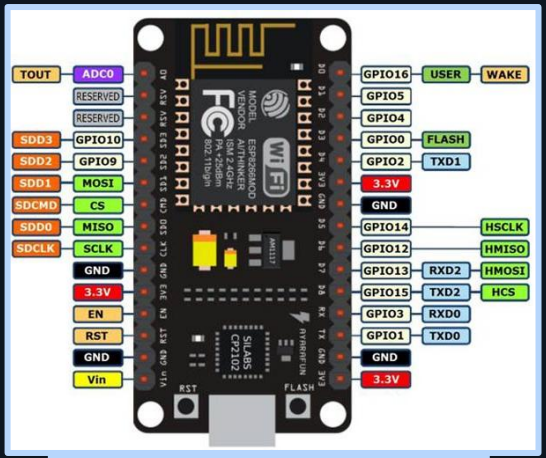
```
  Serial.println();
   if ((char)payload[0]=='1'){
    digitalWrite(ledPin, HIGH);
  }
   else if ((char)payload[0]=='0'){
    digitalWrite(ledPin, LOW);
  }
}

void setup() {
  Serial.begin(115200);
  client.setDebug(true);
  client.wifiConnection(WIFINAME, WIFIPASS);
  client.begin(callback);
  pinMode(ledPin,OUTPUT);
  client.ubidotsSubscribe("potenciometro","virtualled");
}
```

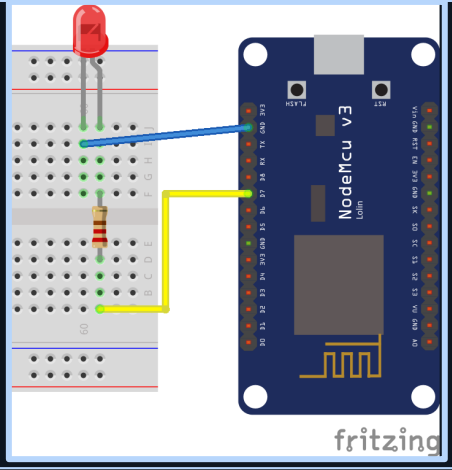




Ubidots

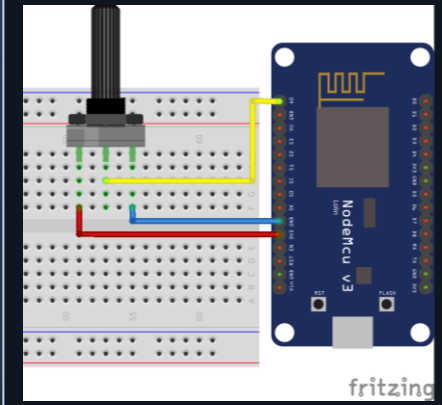
```
void loop() {
   if (!client.connected()) {
   client.reconnect();
   client.ubidotsSubscribe("potenciometro","virtualled");
  }

  client.loop();
}
```

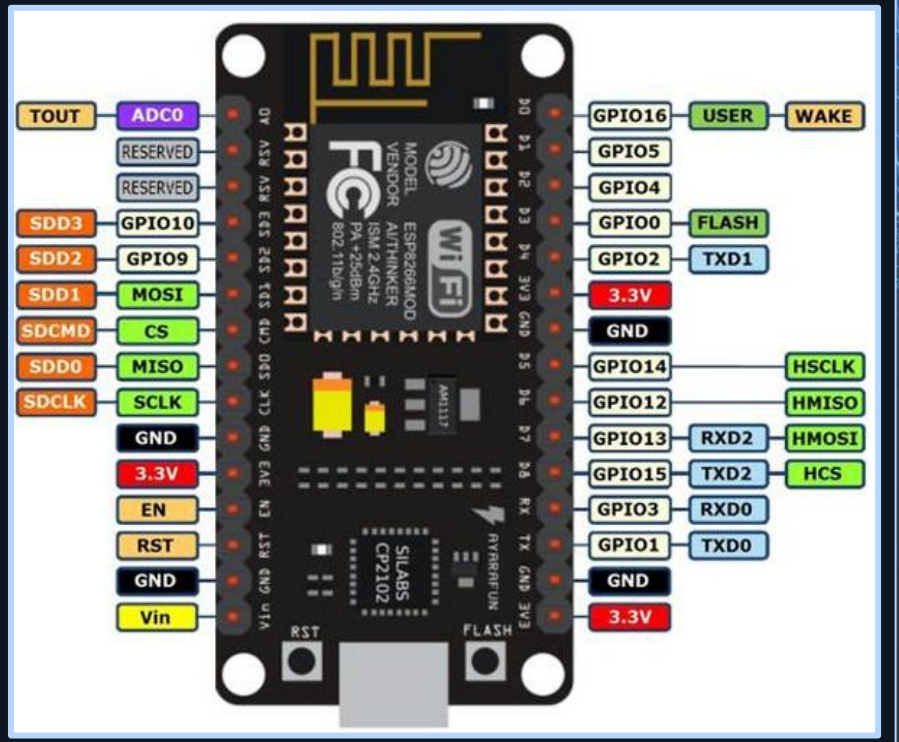




Desafio 01

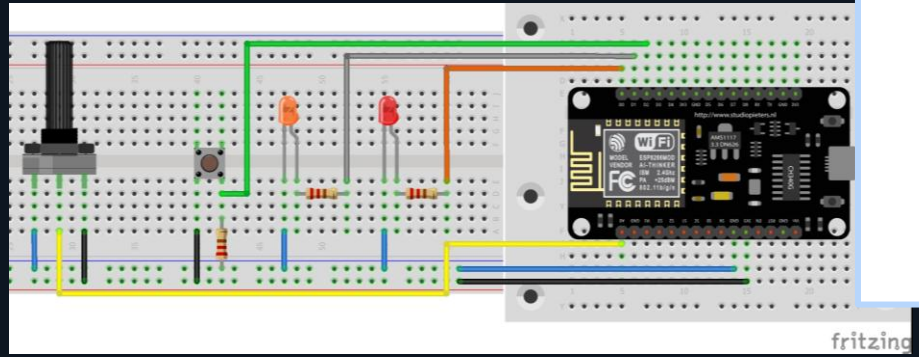
Vamos melhorar ainda mais:
Crie um programa que faça a leitura de 5 em 5 segundo e só mande a informação se o valor atual for maior (atual + 5) ou menor (atual - 5) que o valor anterior.

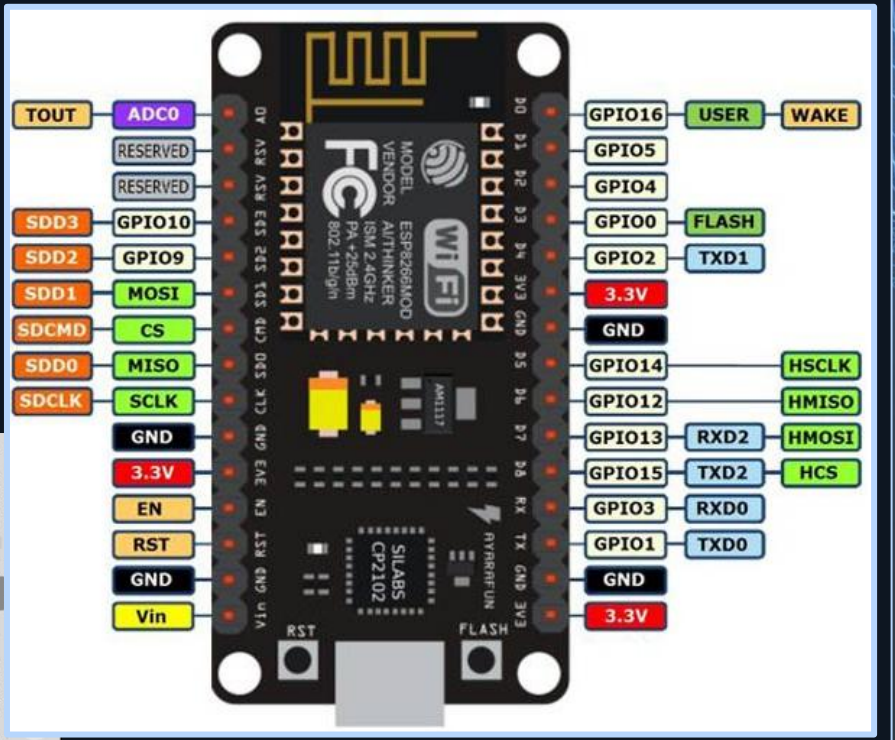




Desafio 02

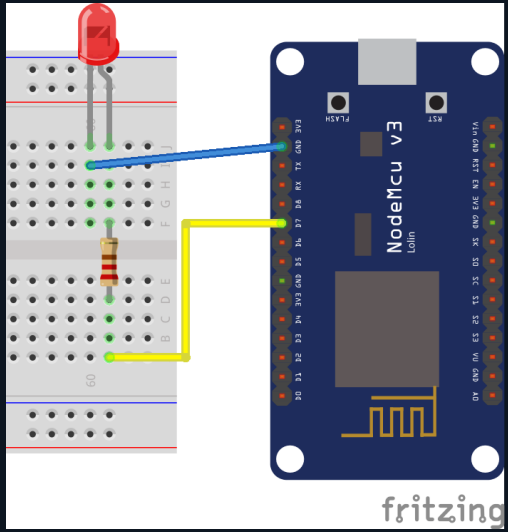
Vamos juntar tudo que aprendemos até aqui e incluir um botão com debounce de LIGA/DESLIGA um LED Virtual, e vamos colocar 2 LEDs um de cada cor.
Faça o código e o Dashboard desse código.

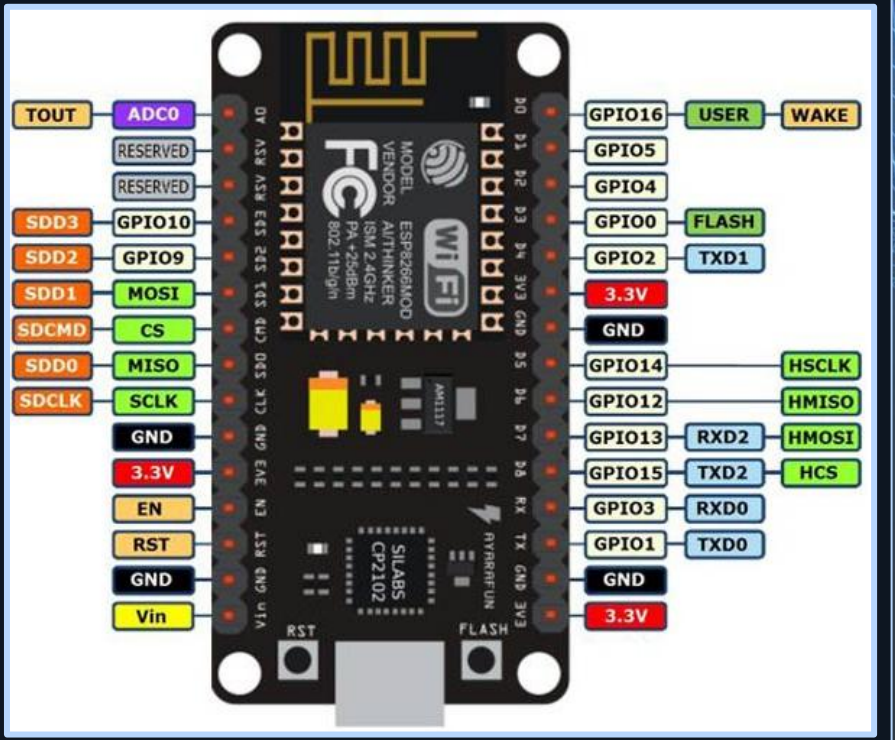




Desafio 03

Faça um código para controlar o LED por sistema PWM, coloque um slidebar no Ubidots.





Obrigado!